

Mock Interview Kit – QE (Playwright + Agentic AI) – LTIMindtree

Role target: Offshore QE, 4–8 yrs, Playwright (TypeScript/Java), REST API testing, SQL, Azure DevOps, AI-assisted & agentic testing

Purpose: Run this as a real mock interview with your referred colleague. Score each section live, then use the scorecard at the end to decide if he's ready.

Update note: C# dropped – he works in TypeScript/Java, so Playwright and API questions are framed for those. AI & Agentic section is expanded and now the heaviest-weighted section, per your priority.

How to use it:

- Ask the question as-is. Let him answer fully before revealing "Ideal Answer Key Points."
- Score 1–5 immediately (don't average later from memory – you'll forget nuance).
- Section 5 (AI & Agentic) carries the most weight and the most questions on purpose – if time is tight, trim other sections before trimming this one.

- The "Red Flags" are things that should make you pause even if the surface answer sounds right.
-

Section 1: Playwright Automation (Weight: 25%)

Q1 (Easy). Walk me through how Playwright's auto-waiting works. Why don't we need

`sleep()` / `Thread.sleep()` ?

- *Key points:* Playwright waits for actionability checks (visible, stable, enabled, receives events) before acting; polls the DOM instead of fixed sleeps; explicit waits only needed for custom conditions (network idle, specific state).
- *Red flag:* If he still reaches for hard sleeps as a first instinct for flakiness.

Q2 (Medium). Explain Page Object Model in your framework. How do you avoid duplicate locators across tests?

- *Key points:* One class per page/component, locators as fields initialized in constructor, methods expose actions not raw locators, base page class for shared elements (header/nav).
- *Red flag:* Locators scattered directly inside test methods.

Q3 (Medium). What's the difference between `Browser` , `BrowserContext` , and `Page` ? Why does context isolation matter for parallel runs?

- *Key points:* `Browser` = process instance; `BrowserContext` = isolated session (cookies, storage, cache) – like an incognito profile; `Page` = a tab within a context. Each test gets its own context so parallel tests don't leak state.

Q4 (Medium). How do you handle authentication once and reuse it across tests without logging in every time?

- *Key points:* `storageState` – save cookies/localStorage after one login, reuse when creating new browser contexts; a global setup step to generate it once before the suite runs.

Q5 (Medium – language specific).

- *If TypeScript:* You have three assertions that each depend on an async API call resolving first. How do you structure this so you're not accidentally racing promises?
- *If Java:* Java's Playwright bindings are synchronous, unlike TS. What does that mean for how you structure waits and parallel test execution compared to a TS/Node suite?
- *Key points (TS):* `await` each call in sequence or `Promise.all` when truly independent, avoid fire-

and-forget async calls before assertions.

- *Key points (Java)*: No async/await — calls block by default; parallelism comes from running multiple test threads/workers, not concurrent promises within one test; thread-safety of shared fixtures matters more.

Q6 (Hard). A test fails intermittently in CI but passes locally 10/10 times. Walk me through your debugging process.

- *Key points*: Check CI vs local differences (headless mode, viewport, network speed, parallel workers colliding on shared data); use trace viewer/video/screenshots on failure; check for race conditions (assertion right after navigation without wait); check test data isolation between parallel tests; retry-and-quarantine as short-term mitigation, not a fix.
- *Red flag*: "I just add a retry and move on" as the *only* answer, with no root-cause step.

Q7 (Hard). How would you test a UI component inside a Shadow DOM or a third-party iframe (e.g., a payment widget)?

- *Key points*: Playwright pierces open shadow DOM automatically with standard locators; for iframes, scope into the frame before locating elements;

closed shadow DOM is harder — may need JS evaluation.

Q8 (Hard). How do you design your suite so smoke, in-sprint, and full regression runs don't all slow each other down?

- *Key points:* Tagging/grouping (`@smoke` , `@regression` , `@sprint`), separate CI pipeline triggers per tag, parallelization/sharding, isolating flaky/slow tests into a nightly-only bucket.
-

Section 2: API Testing (Weight: 15%)

Q9 (Easy). How do you send a POST request with a JSON body and a bearer token?

- *Key points (TS):* Playwright's built-in `APIRequestContext` , or `supertest` / `axios` — set headers, `Authorization: Bearer <token>` , check status code, parse JSON body.
- *Key points (Java):* RestAssured — `given().header()...post()` , or plain `HttpClient` ; check status code and deserialize response.

Q10 (Medium). How do you handle a workflow where Test B needs an auth token or ID generated by Test A?

- *Key points:* Chain requests in a setup/fixture, store token/ID in test-scoped context (not shared mutable global state across parallel tests — that causes race conditions).
- *Red flag:* Using a shared static variable for tokens in a parallel suite without protecting it.

Q11 (Medium). How would you validate that an API response matches an expected schema, not just spot-check a few fields?

- *Key points:* JSON schema validation libraries, or strongly-typed deserialization into a model/interface and asserting required fields are present and correctly typed.

Q12 (Hard). How do you test negative scenarios — invalid payloads, unauthorized access, rate limiting — and what does "good" API coverage look like for a P&C insurance quoting API specifically?

- *Key points:* Boundary/invalid data types, missing required fields, expired/invalid tokens, wrong HTTP verb, concurrency for rate limits; insurance-specific — invalid policy numbers, out-of-range premium calculations, state-specific rating rule violations, malformed effective dates.
-

Section 3: DB Testing – SQL (Weight: 10%)

Q13 (Easy). Write a query to find duplicate policy numbers in a `Policies` table.

- *Key points:* `SELECT PolicyNumber, COUNT(*) FROM Policies GROUP BY PolicyNumber HAVING COUNT(*) > 1 .`

Q14 (Medium). How do you validate backend data after a UI action – say, a claim status update – without just trusting the UI?

- *Key points:* Query the DB directly post-action to confirm status/timestamp/audit fields changed correctly; check related tables (audit log, history), not just the primary record.

Q15 (Hard). You're testing an ETL job migrating policy data between two systems. How do you validate data integrity beyond row counts matching?

- *Key points:* Field-by-field reconciliation, null-handling differences, data type/precision mismatches (decimal rounding on premiums), referential integrity post-migration, sampling + full-diff for critical fields.
-

Section 4: Agile & STLC (Weight: 10%)

Q16 (Easy). What's your role in sprint grooming as a QE, before a single line of code is written?

- *Key points:* Reviewing acceptance criteria for testability, raising edge cases early, estimating test effort, flagging missing NFRs.

Q17 (Medium). How do you decide what gets automated in-sprint vs. deferred to regression?

- *Key points:* Stability of the feature (don't automate UI still changing), business-critical path first, ROI of automation vs. one-off manual check.
-

Section 5: AI & Agentic QE Capabilities – MUST HAVE (Weight: 30%)

This is the JD's explicit differentiator and your priority for this mock. Go deep here – it's where most experienced QEs, even strong automation engineers, are weakest.

A. Foundational AI-Assisted Testing

Q18 (Easy). How have you used AI tools (Copilot, ChatGPT, Claude, etc.) in day-to-day testing? Give one concrete example, not a generic answer.

- *Key points:* Wants specifics — generating boilerplate Page Object scaffolding, drafting test case names from a user story, suggesting locator strategies, explaining a legacy test file. Vague "I use it to write code faster" with no example is weak.

Q19 (Easy). What's the difference between using AI as a "co-pilot" — where it suggests and you decide — versus an "agent" that acts on its own?

- *Key points:* Co-pilot = suggestion only, human approves every action; agent = can take multiple steps and decide next actions itself based on results, with less per-step human approval.

Q20 (Medium). How do you write an effective prompt to generate a test case from a user story? What do you include beyond just pasting the story?

- *Key points:* Role/context (what system, what's already covered), the actual acceptance criteria, constraints (framework, naming conventions, what NOT to test), expected output format. A prompt with zero context produces generic, low-value output.

B. Agentic AI Concepts

Q21 (Medium). What does "tool use" mean for an AI agent — how does it decide when to call a tool (like a test runner or DB query) versus just responding with text?

- *Key points:* The agent is given a set of available tools with descriptions; based on the task and its reasoning, it decides a tool call is needed to get real information or take a real action, rather than guessing.

Q22 (Medium). What's the difference between a single one-shot LLM call and an "agent loop"?

- *Key points:* One-shot = single prompt in, single response out, done. Agent loop = observe result → plan → act → observe again, repeating until the goal is met or it gives up — e.g., an agent that runs a test, sees it fail, inspects the failure, and decides whether to retry, fix, or escalate.

Q23 (Hard). If you were designing an AI agent to triage failing test results automatically, what would it need to do, step by step?

- *Key points:* Read the failure (stack trace/screenshot/logs), classify it (real bug vs. flaky vs. environment issue vs. test itself is wrong), check history (is this test flaky historically?), and either auto-file a defect with

context, retry, or flag for human review — should NOT auto-close or auto-dismiss failures without a clear, defensible reason.

C. Trust & Validation of AI Output

Q24 (Hard). What is a "hallucination" in the context of AI-generated test code, and give a testing-specific example.

- *Key points:* The AI confidently produces something that looks plausible but is factually wrong or doesn't exist — e.g., referencing a locator/selector/API field that was never defined, or inventing an expected value that sounds reasonable but isn't derived from the actual spec.

Q25 (Hard). AI generates expected results for an insurance premium calculation. How do you know if you can trust that output?

- *Key points:* You can't trust it blindly — LLMs can produce confident but wrong numbers for rules-based domain math like rating/premium calc. Expected results for business logic must come from the spec/rating engine/SME, not be generated by the AI. AI is fine for scaffolding, weak as the source of truth for numeric business outcomes.
- *Red flag:* If he'd use the AI's number as the expected result without independent verification.

This is arguably the single most important answer in the whole interview.

Q26 (Hard). How would you design a review checkpoint so AI-generated tests don't silently reduce your suite's real coverage (i.e., tests that always pass regardless of whether there's a bug)?

- *Key points:* Mandatory human review of assertion logic (not just "does it run"), mutation testing or deliberately breaking the feature to confirm the test actually catches it, checking the test isn't asserting something trivially true (like "no error thrown").

D. Applied to Insurance / Enterprise QA

Q27 (Hard). This role touches P&C insurance data. What's your concern about feeding real policyholder data into a public AI tool to help generate test cases?

- *Key points:* PII/PHI exposure, data residency and compliance (state insurance regulations, company data policy), the fix is to use synthetic/masked data for prompts, not real records — awareness of this is a strong positive signal, not knowing it's an issue at all is a real concern for an insurance QE role.

Q28 (Medium). How would you use AI to help build regression coverage for an insurance policy/claims

platform without it just guessing generic web-app patterns?

- *Key points:* Feed it the actual domain rules/acceptance criteria/existing test patterns as context rather than asking it cold; use it to extend patterns from existing well-written tests rather than invent domain logic from scratch.

E. Judgment / Scenario

Q29 (Hard — strong closer). Your manager wants an AI agent to auto-generate AND auto-merge its own tests into the regression suite before a release, to save time. What do you say?

- *Key points:* Pushback with reasoning — auto-merging unreviewed AI output risks false-positive coverage and false confidence exactly when reliability matters most (pre-release). AI-assisted scaffolding is fine; unreviewed AI-authored test *logic* going straight into the suite is not, regardless of time pressure.

Section 6: CI/CD — Azure DevOps (Weight: 10%)

Q30 (Medium). How do you integrate your test suite into an Azure DevOps pipeline and publish results so

they're visible in the build summary?

- *Key points:* YAML pipeline step to install deps/browsers, run tests with a reporter that outputs JUnit/trx/HTML, `PublishTestResults@2` task to surface pass/fail, artifacts for trace/video on failure.

Q31 (Hard). A pipeline run is green, but a bug still slipped to production. How do you investigate whether it's a test gap or a pipeline/environment gap?

- *Key points:* Check if the scenario was covered at all (test gap) vs. covered but the pipeline ran against stale data/wrong environment/skipped stage (pipeline gap); check if the test asserted the wrong thing (false confidence).

Section 7: Scenario & Behavioral — Toughest Round (Bonus, unweighted)

Q32. You inherit a regression suite with 40% flaky tests, 2 weeks before a major release. What's your plan?

- *Key points:* Triage first — quarantine known-flaky rather than let it block the pipeline blindly; prioritize fixing flaky tests on critical business

paths; root-cause common patterns rather than fixing one-by-one; communicate risk honestly rather than hiding it behind a green pipeline.

Q33. Onsite and offshore disagree on whether a defect is a blocker. How do you handle it as the person who found it?

- *Key points:* Present evidence objectively (repro steps, business impact), separate opinion from data, escalate with a recommendation but let the right owner make the call.

Q34. How would you explain to a non-technical stakeholder why "we use AI in testing now" doesn't mean "we need fewer testers" or "zero risk of bugs"?

- *Key points:* AI accelerates scaffolding and coverage generation, but judgment, domain understanding, and validation of correctness still require a human; AI amplifies a good QE's throughput, it doesn't replace the QE's judgment.

Scorecard

Section	Weight	Score (1–5)	Weighted	Notes
Playwright Automation	25%			
API Testing	15%			
DB Testing (SQL)	10%			
Agile & STLC	10%			
AI & Agentic QE (Must-Have)	30%			
CI/CD (Azure DevOps)	10%			
Total	100%		/5	

Scoring guide (per question):

- 5 – Answered correctly with real examples, anticipated follow-ups
- 4 – Correct core answer, minor gaps

- 3 – Right direction, missing key detail or needed prompting
- 2 – Partial/confused, needs significant coaching
- 1 – Incorrect or no answer

Recommendation bands (weighted total):

- 4.0–5.0 → Strong – ready to face the client round as-is
- 3.0–3.9 → Ready with targeted coaching on flagged weak sections
- 2.0–2.9 → Needs 1–2 more prep sessions before you put your name behind him
- Below 2.0 → Not ready yet; be honest with him before it reflects on your referral

One honest note: Section 5 (AI & Agentic, 30% weight, 12 questions) is where most experienced automation QEs – even very strong ones – score lowest, because it tests judgment about AI, not just tool usage. Q25 and Q29 in particular are the ones real interviewers use to separate "has used Copilot" from "actually understands AI in a QA context." Spend the bulk of your coaching time there.